Velocity using the previous position of the node:

$$\mathbf{v} \approx \frac{\mathbf{x}_t - \mathbf{x}_{t-\Delta t}}{\Delta t}$$

Here, $\mathbf{x}_t$ is the current position and $\mathbf{x}_{t-\Delta t}$ is the previous position.

Calculate the distance between two nodes: Suppose the nodes have positions $\mathbf{x}_A$ and $\mathbf{x}_B$. The vector from A to B is:

$$\mathbf{d} = \mathbf{x}_B - \mathbf{x}_A$$

The distance between them is the magnitude of that vector:

$$L = \|\mathbf{d}\| = \sqrt{d_x^2 + d_y^2 + d_z^2}$$

In GLM, it can be calculated with:

```
glm::vec3 delta = nodeB.position - nodeA.position;
float length = glm::length(delta);
```

'delta' tells you both the direction and the separation of the nodes.
'length' gives you their distance.

Calculating the spring force The extension is:

$$\Delta L = L - L_0$$

If ΔL is positive, the beam is stretched. If it is negative, the beam is compressed.
The unit direction vector is:

$$\mathbf{n} = \frac{\mathbf{d}}{L}$$

The spring force applied to node A is:

$$\mathbf{F}_A = k(L - L_0)\mathbf{n}$$

The force on node B is the opposite:

$$\mathbf{F}_B = -\mathbf{F}_A$$

These signs assume **n** points from A to B. When the beam is stretched, node A is pulled towards B and node B is pulled towards A.

You must handle the special case where L is approximately zero before dividing by it.

Add damping The relative velocity is:

$$\mathbf{v}_{rel} = \mathbf{v}_B - \mathbf{v}_A$$

Its component along the beam is:

$$v_{\text{rel}} = (\mathbf{v}_B - \mathbf{v}_A) \cdot \mathbf{n}$$

The damping force on node A is:

$$\mathbf{F}_{d,A} = c\, v_{\text{rel}} \mathbf{n}$$

With this sign convention, the combined force on node A is:

$$\mathbf{F}_A = [k(L - L_0) + c\, v_{\text{rel}}]\, \mathbf{n}$$

Node B receives the negative of this force.

This formulation gives damping that opposes relative motion along the beam. It does not damp motion perpendicular to the beam.

Calculate acceleration Once all forces have been accumulated, use Newton's second law:

$$\mathbf{a} = \frac{\mathbf{F}_{net}}{m}$$

For a node of mass 2 kg experiencing a net force of $(0, -20, 0)$ N:

$$\mathbf{a} = (0, -10, 0) \text{ m/s}^2$$

That is the acceleration vector.

Update velocity and position With semi-implicit Euler integration:

$$\mathbf{v}_{new} = \mathbf{v}_{old} + \mathbf{a}\Delta t$$

$$\mathbf{x}_{new} = \mathbf{x}_{old} + \mathbf{v}_{new}\Delta t$$

In C++, the operations look like:

```
node.velocity += acceleration * dt;
node.position += node.velocity * dt;
```

Here, dt is the timestep in seconds.

If your timestep is $1/120$ seconds, each integration step advances the simulated world by approximately 8.33 milliseconds.

The order matters: update velocity first, then use the new velocity to update position. This is semi-implicit Euler, not the ordinary explicit Euler method.